



Department of Electronics & Communications

25SC2107E – Machine Learning

Skilling Experiment

Diabetes Severity — Multinomial Logistic Regression

Dataset: Pima Indians Diabetes | Course Outcome: CO2 | Environment: VS Code

Prepared by: **Dr. Omprakash Gottam**, Dept. of ECE, KLEF Hyderabad

1 Experiment : Diabetes Severity — Multinomial Logistic Regression

Dataset: Pima Indians Diabetes

Course Outcome: CO2

1.1 Aim

To clean a dataset in which missing values were recorded as zeros, to turn a continuous glucose reading into three severity classes, and to fit a *multinomial* logistic regression that predicts which class a patient falls into. The central lesson is that a single accuracy number can hide a model that has completely given up on two of its three classes.

1.2 Learning Objectives

After completing this experiment, you should be able to:

- Spot impossible zeros in medical data and replace them with `NaN` before doing anything else.
- Turn a number into ordered classes with `pd.cut`, and justify the cut points.
- Explain why the column used to build the target must be removed from the features.
- Read a *per-class* F1 table and say what overall accuracy was hiding.
- Interpret the 3×8 coefficient matrix that multinomial logistic regression produces.

1.3 Environment and Libraries

VS Code with the Python + Jupyter extensions, `.venv` activated.

```
pip install numpy pandas matplotlib scikit-learn
```

1.4 Dataset

Pima Indians Diabetes. Each row is one female patient of Pima Indian heritage, aged 21 or over.

- 768 rows, 8 measurements plus a diabetes diagnosis (**Outcome**).
- **Glucose** is the plasma glucose concentration two hours into an oral glucose tolerance test (OGTT), in mg/dL. **This is the column we will turn into our severity target.**
- The file `diabetes.csv` sits beside the notebook, so the lab runs offline. It can also be fetched with `fetch_openml("diabetes", version=1)`.
- **The dataset is famous for a trap:** missing values were never marked as missing. They were written down as **0**. A blood pressure of 0 or a BMI of 0 is not a measurement, it is an absence.

Note

A third kind of answer

House Pricing Prediction predicted a *number* (regression). Breast Cancer predicted *one of two* labels (binary classification). This experiment predicts *one of three* labels, which is called **multinomial** or multi-class classification.

The practical differences are small but they catch people out:

	Binary (Exp. 5)	Multinomial (this experiment)
<code>model.coef_.shape</code>	1 row of weights	3 rows , one per class
Baseline to beat	the commoner class	the commonest of the three
Measured by	accuracy	accuracy and per-class F1
Failure looks like	guessing one label	silently dropping a class

That last row is the one to watch, and Cell 8 will show it happening.

1.5 Procedure

1. Create a notebook `skill_lab6.ipynb`.
2. Load the data and give the columns readable names.
3. Replace the impossible zeros with NaN and count what is really missing.
4. Cut `Glucose` into three severity classes: Low, Mid and High.
5. Remove `Glucose` from the features, then split 70/15/15 into train, validation and test.
6. Impute the missing values and scale, fitting both on the training set only.
7. Fit a multinomial logistic regression and compare it against the baseline.
8. Build the per-class F1 table across a range of `C` values.
9. Draw one coefficient subplot per class.
10. Choose `C` on the validation set, then report once on the test set.
11. Write `multinomial_analysis()` returning `[C, overall_acc, F1_Low, F1_Mid, F1_High]`.

1.6 Notebook Implementation

Cell 1 — Import the libraries

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split
5 from sklearn.impute import SimpleImputer
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.linear_model import LogisticRegression
8 from sklearn.metrics import accuracy_score, f1_score,
   classification_report
9 from sklearn.metrics import confusion_matrix

```

- `SimpleImputer` is new. Experiments House Pricing Prediction and Breast Cancer classification used datasets with nothing missing, so this is the first time you need it.
- `f1_score` is the important import. Accuracy alone will mislead you in this experiment, and F1 is what exposes the problem.

Cell 2 — Load the data

```

1 diabetes = pd.read_csv("diabetes.csv")
2 print(diabetes.shape)
3 print(diabetes.isna().sum().sum(), "values look missing")
4 diabetes.head()

```

- 768 rows and 9 columns.
- **Pandas reports zero missing values. Do not believe it.** Nothing is marked NaN, so every check you know how to run will say the data is clean. The next cell shows that it is not.

Cell 3 — The zeros that are really missing values

```

1 suspect = ["Glucose", "BloodPressure", "SkinThickness", "Insulin",
   "BMI"]
2 for col in suspect:
3     n = (diabetes[col] == 0).sum()
4     print("%-15s %3d zeros  (%.1f%%)" % (col, n, 100 * n /
   len(diabetes)))
5
6 diabetes[suspect] = diabetes[suspect].replace(0, np.nan)
7 print()
8 print(diabetes.isna().sum())

```

- Ask of each column: *is zero a possible reading for a living patient?* A BMI of 0, a blood pressure of 0 and a glucose of 0 are all impossible. So those zeros mean “not recorded”.
- **Pregnancies** is deliberately *not* in the list. Zero pregnancies is a perfectly real value. **You cannot apply this fix column by column without thinking** — that is the whole skill here.
- The true picture is bad: **Insulin is 48.7% missing** and **SkinThickness** is 29.6% missing. **BloodPressure** 35, **BMI** 11, **Glucose** 5.
- Had you skipped this step, the model would have treated hundreds of patients as having an insulin level of zero, and every coefficient you read afterwards would have been meaningless.

Cell 4 — Build the severity target

```

1 diabetes = diabetes.dropna(subset=["Glucose"]).reset_index(drop=True)
2
3 print("Glucose range:", diabetes["Glucose"].min(), "to",
4       diabetes["Glucose"].max())
5
6 diabetes["Severity"] = pd.cut(diabetes["Glucose"],
7                               bins=[0, 99.9, 139.9, 1000],
8                               labels=["Low", "Mid", "High"])
9 print(diabetes["Severity"].value_counts().sort_index())

```

- The 5 patients whose **Glucose** is missing are **dropped, not imputed**. You may guess a missing feature, but you must never invent the answer you are trying to predict. That leaves 763 patients.
- **The cut points are clinical, not arbitrary.** For a 2-hour OGTT, under 100 mg/dL is normal, and 140 and above is the recognised threshold for impaired glucose tolerance. So Low is normal, Mid is raised but under the threshold, and High is at or past it.
- **Look at the maximum: 199.** The true clinical marker for diabetes is 200, and not one patient exceeds 199. The values were capped, exactly like the \$500,000 cap in House Pricing Prediction. Had we used 200 as a cut point, the High class would have been empty.
- The classes come out at **192 Low, 374 Mid, 197 High**. Mid is about twice either of the others. That imbalance is mild, and it is precisely what makes accuracy untrustworthy later.

Cell 5 — Remove the leak, then split 70/15/15

```

1 X = diabetes.drop(columns=["Glucose", "Severity"])
2 y = diabetes["Severity"].astype(str)
3
4 X_train, X_temp, y_train, y_temp = train_test_split(
5     X, y, test_size=0.30, random_state=42, stratify=y)
6 X_val, X_test, y_val, y_test = train_test_split(
7     X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp)
8
9 print(X_train.shape, X_val.shape, X_test.shape)
10 print(X.columns.tolist())

```

- **Dropping Glucose is the most important line in this notebook.** Severity was cut directly from Glucose, so leaving it in would let the model read the answer off its own input. You would score close to 100% and have learned nothing. That mistake is called **target leakage**.
- What remains is a genuinely useful question: can the other measurements — insulin, BMI, age, blood pressure and the rest — tell us roughly where a patient’s glucose sits, without running the glucose test?
- **Three splits, not two**, matching the course convention: 534 train, 114 validation, 115

test. You tune **C** on the validation set and touch the test set once, at the very end. If you pick **C** using the test set, your test score stops being an honest estimate.

- **stratify** is applied at both splits so all three parts keep the 25/49/26 class mix.

Cell 6 — Impute, then scale

```
1 imputer = SimpleImputer(strategy="median")
2 scaler = StandardScaler()
3
4 X_train_s = scaler.fit_transform(imputer.fit_transform(X_train))
5 X_val_s = scaler.transform(imputer.transform(X_val))
6 X_test_s = scaler.transform(imputer.transform(X_test))
7
8 print(pd.Series(imputer.statistics_,
9               index=X.columns).round(1).to_string())
```

- **Median, not mean.** Insulin is strongly right-skewed, and a handful of very high readings would drag the mean upwards. The median is unmoved by them.
- **fit.transform on train, transform on validation and test.** The median is computed from the training patients alone. Computing it over the whole dataset would let information from the test set seep into training — a second, subtler kind of leakage.
- Imputing first and scaling second is the right order: **StandardScaler** cannot handle NaN.
- Be honest about what this costs. Nearly half the insulin values are now a single repeated number, which weakens that column. Reporting that limitation is part of the result.

Cell 7 — Baseline, then the first multinomial fit

```
1 print("always-Mid baseline:", round((y_val == "Mid").mean(), 4))
2
3 model = LogisticRegression(C=1, max_iter=5000, random_state=42)
4 model.fit(X_train_s, y_train)
5
6 pred = model.predict(X_val_s)
7 print("validation accuracy:", round(accuracy_score(y_val, pred), 4))
8 print("classes_:", model.classes_)
9 print("coef_ shape:", model.coef_.shape)
```

- The baseline is **0.4912**: always answering “Mid” is right about half the time. The model reaches **0.6316**, so it has genuinely learned something — but a 63% classifier is not a clinical instrument, and the honest framing is that these features carry only a moderate signal about glucose.
- **coef_ has shape (3, 8): three rows of eight weights.** Binary logistic regression gives one row. Multinomials gives one row per class, each answering “how much does each feature push a patient towards *this* class?”
- **Read classes_ carefully. It prints ['High' 'Low' 'Mid']** — alphabetical order, not the Low/Mid/High order you defined. Row 0 of **coef_** is *High*, not Low. Assuming your own order here is one of the easiest ways to publish a completely wrong interpretation.

Note

multi_class="multinomial" no longer exists

Older tutorials, and most answers you will find online, write this:

```
1 model = LogisticRegression(multi_class="multinomial") # fails
```

In current scikit-learn that raises **TypeError: __init__() got an unexpected keyword argument 'multi_class'**. The argument was deprecated and has now been removed.

You do not need it. When `y` has more than two classes, `LogisticRegression` fits a multinomial model automatically. Just leave the argument out. If you copy code from a tutorial and hit this error, delete the argument rather than downgrading scikit-learn.

Cell 8 — Per-class F1 against C, and what accuracy was hiding

```

1 class_names = ["Low", "Mid", "High"]
2 rows = []
3
4 for C in [0.001, 0.01, 0.1, 1, 10, 100]:
5     m = LogisticRegression(C=C, max_iter=5000, random_state=42)
6     m.fit(X_train_s, y_train)
7     p = m.predict(X_val_s)
8     f1s = f1_score(y_val, p, average=None, labels=class_names,
9                  zero_division=0)
10    rows.append({"C": C, "overall_acc": accuracy_score(y_val, p),
11               "F1_Low": f1s[0], "F1_Mid": f1s[1], "F1_High":
12                  f1s[2]})
13
14 pd.DataFrame(rows).round(4)

```

- **Look at the first row, $C = 0.001$.** Overall accuracy is 0.4912, which sounds like a weak-but-working model. It is not. `F1_Low` and `F1_High` are both **exactly 0.0000**.
- That model answers “Mid” for every single patient. It never once says Low or High. It scores 49% only because Mid really is 49% of the data — **it has exactly matched the baseline while appearing to be a trained model.**
- **This is the point of the whole experiment. One accuracy number cannot tell the difference between a model that has learned three classes and a model that has abandoned two of them.** The per-class column can, instantly.
- Follow the columns downwards as the penalty weakens. `F1_High` climbs 0.0000, 0.4681, 0.5455, 0.5965 and then stops. `F1_Low` climbs 0.0000, 0.1250, 0.6250, 0.5957. The minority classes are the last to come back and the first to be sacrificed — when a penalty forces the model to economise, it economises on the small classes first.
- `labels=class_names` is essential. Without it `f1_score` returns the scores in alphabetical order and your three columns would be silently mislabelled.
- `zero_division=0` tells scikit-learn to report 0.0 rather than warn when a class is never predicted. Here, that zero is the finding.

Cell 9 — One coefficient subplot per class

```

1 best = LogisticRegression(C=1, max_iter=5000, random_state=42)
2 best.fit(X_train_s, y_train)
3
4 order = list(best.classes_)
5 fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True)
6
7 for ax, cls in zip(axes, class_names):
8     row = best.coef_[order.index(cls)]
9     ax.bar(X.columns, row, color="steelblue")
10    ax.axhline(0, color="black", linewidth=0.8)
11    ax.set_title("Pushes towards: " + cls)
12    ax.tick_params(axis="x", rotation=90)
13
14 axes[0].set_ylabel("coefficient")
15 plt.tight_layout()
16 plt.show()

```

- `order.index(cls)` looks the row up by name instead of trusting the position. Given that `classes_` is alphabetical, hard-coding `coef_[0]` for Low would plot the High weights under a Low heading.
- **Insulin dominates every panel**, and its sign flips across them: about -1.38 for Low and $+1.00$ for High. High insulin pushes hard towards the High class and hard away from Low, which is exactly the physiology you would expect, since insulin rises with glucose.
- **Outcome** (the diabetes diagnosis) is the second strongest, again with opposite signs on Low and High.
- **The three rows are competing, not independent.** Multinomial logistic regression scores all three classes and takes the largest, so what matters is each feature's weight in one panel *relative* to the others. This is why the Mid panel looks so flat: Mid wins by default when neither Low nor High makes a strong case.
- `sharey=True` puts all three panels on one scale. Without it, the flat Mid panel would be stretched to look as dramatic as the others.

Cell 10 — Pick C on validation, report once on test

```

1 final = LogisticRegression(C=1, max_iter=5000, random_state=42)
2 final.fit(X_train_s, y_train)
3 test_pred = final.predict(X_test_s)
4
5 print("baseline (always Mid):", round((y_test == "Mid").mean(), 4))
6 print("TEST accuracy          :", round(accuracy_score(y_test,
7               test_pred), 4))
8 print()
9 print(classification_report(y_test, test_pred, labels=class_names,
10               zero_division=0))
11 print(confusion_matrix(y_test, test_pred, labels=class_names))

```

- $C = 1$ was the best validation score in Cell 8, so it is the one value carried through to the test set.
- **Test accuracy is 0.6000, against 0.6316 on validation.** The drop is normal and worth stating plainly: you chose C by looking at the validation set, so that score is mildly optimistic. **The test number is the one you report.**
- Against a 0.4870 baseline, 0.6000 is a real gain of about 11 points — modest, honest, and roughly what these features can support without the glucose reading itself.
- In the confusion matrix, look at the Low–High corners. Confusing Low with Mid is a small error; confusing **Low with High** means missing a patient at genuine clinical risk. Ordered classes make some mistakes worse than others, and a plain accuracy score treats them all alike.

Deliverable Function

```

1 def multinomial_analysis(X_train, y_train, X_val, y_val,
2               class_names, C_values):
3     rows = []
4
5     for C in C_values:
6         model = LogisticRegression(C=C, max_iter=5000,
7               random_state=42)
8         model.fit(X_train, y_train)
9         pred = model.predict(X_val)
10
11         f1s = f1_score(y_val, pred, average=None, labels=class_names,
12               zero_division=0)

```



```

13         row = {"C": C, "overall_acc": accuracy_score(y_val, pred)}
14         for name, score in zip(class_names, f1s):
15             row["F1_" + name] = score
16         rows.append(row)
17
18     return pd.DataFrame(rows).round(4)
19
20
21 results = multinomial_analysis(X_train_s, y_train, X_val_s, y_val,
22                               ["Low", "Mid", "High"],
23                               [0.001, 0.01, 0.1, 1, 10, 100])
24 results

```

- The required columns are [C, overall_acc, F1_Low, F1_Mid, F1_High]. Building them with "F1_" + name in a loop means the function still works for four or five classes without a rewrite — do not hard-code the three names.
- class_names is passed in and handed straight to labels=, so the caller's order is the column order. This is what keeps the labels honest.
- **How to read the finished table:** scan overall_acc for the best row, then check that row's three F1 columns are all non-zero. A high accuracy sitting beside a 0.0000 is not a good model, it is a warning.
- The bottom three rows are identical. Beyond C = 1 the penalty is so weak it no longer binds, so all larger values give the same fit. With 534 rows and only 8 features there is little to overfit — the same situation as House Pricing Prediction, and the opposite of Breast cancer classification.

1.7 Expected Output

```

(768, 9)
0 values look missing

Glucose      5 zeros (0.7%)
BloodPressure 35 zeros (4.6%)
SkinThickness 227 zeros (29.6%)
Insulin      374 zeros (48.7%)
BMI          11 zeros (1.4%)

Glucose range: 44.0 to 199.0
Low      192
Mid      374
High     197

(534, 8) (114, 8) (115, 8)
always-Mid baseline: 0.4912
validation accuracy: 0.6316
classes_: ['High' 'Low' 'Mid']
coef_ shape: (3, 8)

   C  overall_acc  F1_Low  F1_Mid  F1_High
0.001      0.4912  0.0000  0.6588  0.0000
0.010      0.5351  0.1250  0.6443  0.4681
0.100      0.6228  0.6250  0.6560  0.5455
1.000      0.6316  0.5957  0.6613  0.5965
10.000     0.6316  0.5957  0.6613  0.5965
100.000    0.6316  0.5957  0.6613  0.5965

baseline (always Mid): 0.487

```


TEST accuracy	: 0.6
---------------	-------

1.8 Result

The 768-row Pima Indians Diabetes dataset was cleaned by replacing impossible zeros with NaN, which revealed that **Insulin** was 48.7% missing and **SkinThickness** 29.6% missing. The 5 patients with no glucose reading were dropped, leaving 763. Glucose was cut at the clinical thresholds of 100 and 140 mg/dL into Low, Mid and High severity classes of 192, 374 and 197 patients. After removing **Glucose** to prevent target leakage, a 70/15/15 split gave 534 training, 114 validation and 115 test patients, with median imputation and standardisation both fitted on the training set alone. A multinomial logistic regression produced a 3×8 coefficient matrix and reached a validation accuracy of 0.6316 at $C = 1$ against an always-Mid baseline of 0.4912, and a test accuracy of 0.6000 against a baseline of 0.4870. At $C = 0.001$ the model collapsed to predicting Mid for every patient, scoring 0.4912 overall while returning an F1 of exactly 0.0000 for both Low and High.

Note

What to take away

- **Zeros are not always zeros.** Check whether each value is physically possible before you trust a “no missing values” report. Judge it column by column: zero pregnancies is real, zero BMI is not.
- **Never leave the source column in.** If the target was built from **Glucose**, **Glucose** must go. A suspiciously excellent score is usually leakage, not skill.
- **Overall accuracy cannot see a dropped class.** A model predicting only the majority class scored 0.4912 here and looked plausible. Per-class F1 exposed it in one glance. **Whenever your classes are uneven, report per-class scores.**
- **Check classes_ before reading any coefficient.** It is alphabetical, so it will rarely match the order you have in your head.

One limitation worth stating in your viva: Low, Mid and High are *ordered*, but multinomial logistic regression does not know that. It treats them as three unrelated names, so predicting High for a Low patient costs it exactly as much as predicting Mid. Ordinal regression is the tool that uses the ordering, and it is the natural next step from here.